

Practical 5

AIM: To Implementation of a knapsack problem using dynamic programming

1 ALGORITHM:

```
ALGORITHM
Input:
    n → no. of items
    w → capacity of knapsack
    w[1...n] → weight of each item
    P[1...n] → profit of each item
Output: Maximum profit that can be obtained
Step 1: start
Step 2: For i = 0 to n
        K[i][0] = 0
Step 3: For j = 0 to w
        K[0][j] = 0
Step 4: For i = 1 to n for j = 1 to w
        If w[i] ≤ j, then
            K[i][j] = max(P[i] + K[i-1][j-w[i]], K[i-1][j])
        Else,
            K[i][j] = K[i-1][j]
Step 5: Maximum profit = K[n][w]
Step 6: Print maximum profit
Step 7: stop
```

2 PROGRAM:

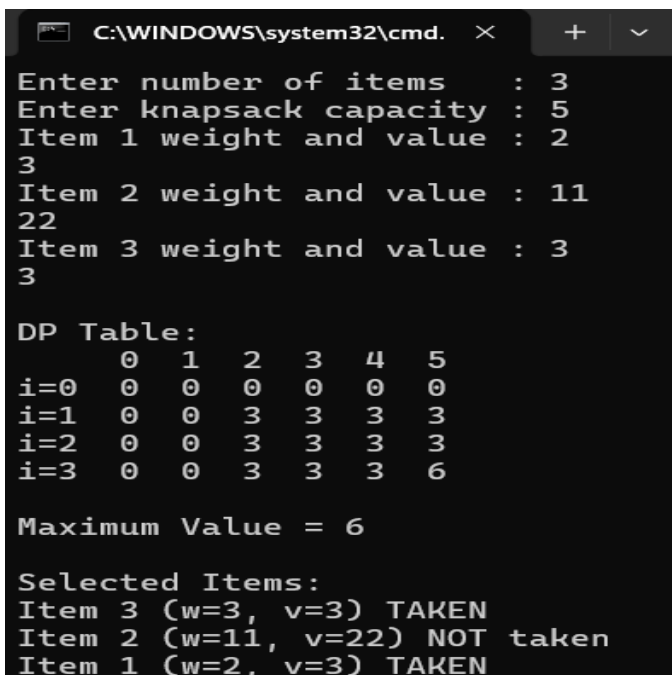
```
#include <iostream>
#include <vector>
using namespace std;
class Knapsack {
private:
    vector<int> w; // weights
    vector<int> v;
    int n, W;
    vector<vector<int>> dp;
public:
    void getInput() {
        cout << "Enter number of items : ";
        cin >> n;
        cout << "Enter knapsack capacity : ";
        cin >> W;
        w.resize(n + 1); v.resize(n + 1);
        dp.assign(n + 1, vector<int>(W + 1, 0));
        for (int i = 1; i <= n; i++) {
            cout << "Item " << i << " weight and value : ";
```

```

    cin >> w[i] >> v[i];}
void solve() {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= W; j++) {
            if (w[i] <= j) {
                dp[i][j] = max(v[i] + dp[i - 1][j - w[i]], dp[i - 1][j]);
            } else {dp[i][j] = dp[i - 1][j];}}
    }
void printTable() {
    cout << "\nDP Table:\n ";
    for (int j = 0; j <= W; j++){cout << j << " ";}
    cout << endl;
    for (int i = 0; i <= n; i++) {
        cout << "i=" << i << " ";
        for (int j = 0; j <= W; j++)
            cout << dp[i][j] << " ";
        cout << endl;}}
void findItems() {
    cout << "\nSelected Items:\n";int j = W;
    for (int i = n; i >= 1; i--) {
        if (dp[i][j] != dp[i - 1][j]) {
            cout << "Item " << i << " (w=" << w[i] << ", v=" << v[i] << ")TAKEN\n";
            j -= w[i];} else {
    cout << "Item " << i << " (w=" << w[i] << ", v=" << v[i] << ")NOTtaken\n";}}}
void showResult() {
    cout << "\nMaximum Value = " << dp[n][W] << endl;}};
int main() {
    Knapsack k;
    k.getInput();k.solve();k.printTable();k.showResult();k.findItems();
    return 0;}

```

3 OUTPUT:



```

C:\WINDOWS\system32\cmd.  X  +  v
Enter number of items      : 3
Enter knapsack capacity    : 5
Item 1 weight and value   : 2
3
Item 2 weight and value   : 11
22
Item 3 weight and value   : 3
3

DP Table:
   0  1  2  3  4  5
i=0  0  0  0  0  0  0
i=1  0  0  3  3  3  3
i=2  0  0  3  3  3  3
i=3  0  0  3  3  3  6

Maximum Value = 6

Selected Items:
Item 3 (w=3, v=3) TAKEN
Item 2 (w=11, v=22) NOT taken
Item 1 (w=2, v=3) TAKEN

```

4 TIME COMPLEXITY:

Time complexity

For $i = 1$ to n

For $j = 1$ to w

If $w[i] \leq j$

$K[i][j] = \max(P[i] + K[i-1][j-w[i]], K[i-1][j])$

Else

$K[i][j] = K[i-1][j]$ so,

$T = n \times w \times O(1)$

Ignoring constants:

$$T = nw$$

Suppose:

$n = 4$ items, $w = 5$ capacity

No. of table calculations:

$$n \times w = 4 \times 5 = 20$$

Worst case:

Every table cell is calculated.

$$T_{\text{worst}} = n \times w$$

If $n = 4$, $w = 5$

$$T_{\text{worst}} = 4 \times 5 = 20 \text{ Therefore,,}$$

Worst case = $O(nw)$

Average case:

The algorithm still goes through the $n \times w$ table cells.

Therefore,

Average case = $O(nw)$

5 Conclusion:

The 0/1 Knapsack algorithm finds the combination of items having maximum profit without exceeding the given knapsack capacity